

Guide de la Phase de Démonstration — 4 minutes

RansomShield

Système de détection et de réponse aux attaques ransomware

Loïc SANGRONIO | IFRI-UAC | Licence Sécurité Informatique | 11 juillet 2026

#	Ce qu'on montre	Durée	Résultat attendu
1	Tableau de bord + agents actifs	40 s	Vue globale de la console SOC
2	Attaque réelle depuis une VM	1 min	Événements injectés en SSH
3	Détection en temps réel	50 s	Alerte critique + incident auto
4	Réponse SOC + timeline d'audit	55 s	Approbation + rollback + audit
5	Simulation kill chain complète	35 s	22 événements, pipeline validé
Total		4 min	

1 Préparation (5 minutes avant la démo)

✓ Ce qu'il faut vérifier avant de commencer

- ✓ Vérifier que le serveur SOC est actif :

```
curl -s http://localhost:8000 | head -5
# Si rien ne répond -- relancer AVEC l'option -host=0.0.0.0 (obligatoire pour
les VMs) :
cd /home/sangronio/ransomshield/backend-laravel
php artisan serve -host=0.0.0.0 -port=8000
```

- ✓ Vérifier que les 3 VMs sont actives :

```
virsh list -all # Résultat attendu : rs-client-1, rs-fileserver, rs-attacker en
"running"
```

- ✓ Vérifier que l'agent est actif sur la VM victime :

```
ssh ubuntu@10.20.0.81 # mot de passe : ransomshield2026
sudo systemctl status ransomshield-agent # doit afficher "active (running)"
sudo journalctl -u ransomshield-agent -n 5 -no-pager # doit afficher
"[HEARTBEAT] OK"
```

- ✓ Préparer la VM victime (fichiers propres pour la démo) :

```
ssh ubuntu@10.20.0.81
rm -f /home/ubuntu/Documents/*.locked /home/ubuntu/HOW_TO_DECRYPT.txt
ls /home/ubuntu/Documents/ # doit être vide (ou contenir uniquement des .docx)
```

- ✓ Ouvrir le navigateur sur <http://localhost:8000/console/dashboard> et se connecter :
 - E-mail : `admin@ransomshield.local`
 - Mot de passe + code 2FA TOTP (application Google Authenticator)
- ✓ Ouvrir deux fenêtres côte à côte : fenêtre 1 = navigateur SOC | fenêtre 2 = terminal SSH (connexion VM).
- ✓ Nettoyer les événements de test précédents dans la console si nécessaire (filtrer par date ou niveau pour une démonstration propre).

⚠ Si un agent VM est hors ligne

Pré-requis : le SOC doit être lancé avec `-host=0.0.0.0` (pas `127.0.0.1`).

```
ssh ubuntu@10.20.0.81 # mot de passe : ransomshield2026
sudo systemctl restart ransomshield-agent
sudo systemctl status ransomshield-agent # doit afficher "active (running)"
sudo journalctl -u ransomshield-agent -f # surveiller les [HEARTBEAT] OK
```

Vérifier ensuite dans la console que le heartbeat se met à jour (délai max : 30 s).

2 Déroulé étape par étape



Étape 1

0 :00 – 0 :40 (40 secondes)

1. Aller sur `/console/dashboard`.
2. Montrer les compteurs en haut : alertes actives, incidents en cours, agents critiques, actions en attente.
3. Montrer le graphique d'événements (mis à jour toutes les 30 s).
4. Aller sur `/console/agents`.
5. Montrer les 3 VMs actives avec leur IP, heartbeat et niveau de risque :

<code>rs-client-1</code> (10.20.0.81)	<code>rs-fileserver</code> (10.20.0.38)	<code>rs-attacker</code> (10.20.0.72).
---------------------------------------	---	--

« Voici la console SOC de RansomShield — elle centralise la supervision de tous les terminaux. Les compteurs en haut donnent l'état en temps réel : alertes actives, incidents, agents critiques, actions en attente. Trois agents Python tournent sur trois machines virtuelles KVM en réseau isolé, chacun avec un heartbeat toutes les 30 secondes. »



Étape 2 — Attaque réelle depuis une VM

0 :40 – 1 :40 (1 minute)

Basculer sur le terminal SSH (fenêtre 2) et jouer les commandes suivantes :

```
# 1. Se connecter à la VM victime
ssh ubuntu@10.20.0.81 # mot de passe : ransomshield2026

# 2. Créer un fichier document normal
touch /home/ubuntu/Documents/rapport_demo.docx
sleep 2

# 3. Chiffrement simulé : renommer avec extension .locked
# → déclenche file_encrypted_extension (score 135, niveau Critique)
mv /home/ubuntu/Documents/rapport_demo.docx /home/ubuntu/Documents/rapport_demo.docx.locked
sleep 2

# 4. Déposer une note de rançon (fichier absent → ransom_note_created)
echo "Your files are encrypted. Pay 1 BTC." > /home/ubuntu/HOW_TO_DECRYPT.txt
```

Pour confirmer que l'agent envoie bien les événements :

```
# Dans un 3e terminal (optionnel, utile si la console tarde)
ssh ubuntu@10.20.0.81
sudo journalctl -u ransomshield-agent -f
# Vous devez voir : [EVENT] file_encrypted_extension ... risk=critical score=135
```

« Je me connecte en SSH sur la VM victime et simule un chiffrement : renommage avec l'extension `.locked`, puis dépôt d'une note de rançon. L'agent Python surveille ce dossier via Watchdog et inotify. Ces événements sont placés dans la file locale SQLite, puis transmis au SOC via l'API sécurisée. »

🕒 Étape 3 — Détection en temps réel

1 :40 – 2 :30 (50 secondes)

Revenir sur le navigateur (fenêtre 1) — ne pas rafraîchir manuellement.

1. `/console/events` — l'événement `file_encrypted_extension` de `rs-client-1` est apparu avec un score de **135** et le niveau **Critique**.
2. `/console/alerts` — l'alerte critique a été générée automatiquement avec ses signaux (extension suspecte + note de rançon) et l'agent source.
3. `/console/incidents` — l'incident a été ouvert automatiquement. Cliquer dessus pour afficher la fiche : alertes rattachées, signaux, actions proposées.

« En moins de 5 secondes après le renommage sur la machine virtuelle, le moteur de détection a calculé un score de 135 à partir des signaux cumulés : +80 pour l'extension suspecte, +55 pour la note de rançon. Ce score dépasse le seuil critique configuré à 100. Une alerte a été générée et un incident de sécurité a été ouvert automatiquement dans la console. »

⇒ **Score 135 → Niveau Critique → Alerte + Incident ouverts automatiquement en < 5 s**

🕒 Étape 4 — Réponse SOC et timeline d'audit

2 :30 – 3 :25 (55 secondes)

1. `/console/protection-actions` — montrer la file d'approbation : actions proposées par le moteur (scan actif, isolation réseau).
2. Cliquer **Approuver** sur l'action `scan actif` : l'agent reçoit la commande et exécute le scan.
3. Si l'action "isolation réseau" est disponible : l'approuver, puis montrer le bouton **Rollback** pour lever l'isolation.
4. `/console/incidents/{id}/timeline` — afficher la timeline chronologique : événement reçu → alerte créée → incident ouvert → action proposée → action approuvée.

« Dans RansomShield, le moteur propose — l'opérateur décide. J'approuve l'action de scan : l'agent reçoit la commande et l'exécute. Si j'avais isolé la machine, le bouton rollback permet de lever cette isolation en un clic. Chaque décision est horodatée dans la timeline : de la détection à la réponse, tout est tracé. »

⇒ **Timeline complète : de la détection à la décision, tout est tracé et horodaté**

**Étape 5 — Simulation de la kill chain complète**

3 :25 – 4 :00 (35 secondes)

1. Aller sur `/console/simulation`.
2. Sélectionner l'agent cible : **rs-client-1**.
3. Choisir le scénario "**Kill chain complète**" (22 événements).
4. Cliquer **Lancer la simulation**.
5. Observer la cascade d'alertes sur `/console/alerts`.

« Le simulateur injecte 22 événements dans le même pipeline que pour une vraie attaque. Tous sont marqués `is_simulation=true` pour un nettoyage facile. Cela démontre la résilience du moteur face à un volume élevé d'événements, sans toucher l'environnement réel. »

⇒ **22 événements traités, aucun perdu : la résilience de la file SQLite est démontrée**

3 Récapitulatif des capacités démontrées**✓ Ce que le jury a pu observer**

Capacité	Prouvée par	Étape
Multi-agents sur VMs réelles	3 agents actifs en console	1
Surveillance en temps réel	Événement détecté en < 5 s	2–3
Analyse comportementale	Score 135, niveau critique auto	3
Incident auto-ouvert	Fiche incident avec signaux	3
Contrôle humain	File d'approbation (approuver/rejeter)	4
Rollback	Isolation réseau levée en un clic	4
Timeline d'audit complète	Chronologie horodatée	4
Résilience du pipeline	22 événements sans perte	5

4 Plan de secours (si quelque chose échoue)**⚠ La VM ne répond pas en SSH — injection directe depuis le poste**

Injecter un événement directement via `curl` depuis le poste hôte :

```
curl -s -X POST http://127.0.0.1:8000/api/agent/events \
-H "Content-Type: application/json" \
-H "X-Agent-Secret: 3cQvlo6vlnn0H20PKzm0qumhfvQy3wRi1iG45CsaTrq3DlbbwQzxXINqIerH8q8p" \
-d '{"agent_uuid":"ed71a368-13e3-4776-af73-3750dd44b6b7",
"event_type":"file_encrypted_extension",
"file_path":"/home/ubuntu/Documents/rapport_demo.docx.locked",
"file_extension":"locked","file_name":"rapport_demo.docx.locked"}'
```

La réponse JSON confirme : `{"risk_level":"critical","score":135,...}`.

L'alerte et l'incident sont créés automatiquement — passer directement à l'**étape 3**.

*Note : l'UUID ci-dessus est celui de l'agent **rs-client-1** (persisté dans son state local).*

⚠ Le navigateur ne montre pas les nouvelles alertes

Forcer un rafraîchissement manuel (F5) ou aller directement sur `/console/events` et filtrer par niveau “Critique”.

⚠ La simulation ne se lance pas

Vérifier qu’un agent **actif** (heartbeat < 2 min) est sélectionné. Utiliser l’agent local DESKTOP-TEST01 en dernier recours.

Document interne — soutenance RansomShield
Loïc SANGRONIO — IFRI-UAC — 11 juillet 2026